Dev Command Cheatsheet

Updated 2026-02-25

DOCKER: one-offs

```
docker run --rm -it IMAGE:TAG bash
docker run --rm -v "$PWD":/w -w /w IMAGE:TAG CMD
docker run --rm -e KEY=VAL IMAGE:TAG CMD
docker run --rm --user "$(id -u):$(id -g)" -v
  "$PWD":/w -w /w IMAGE CMD
docker run --rm -p 5432:5432 postgres:16
docker exec -it CONTAINER sh
docker logs -f --tail=200 CONTAINER
docker cp ./file CONTAINER:/path/
docker system df
docker system prune -af  # destructive
```

DOCKER COMPOSE

```
docker compose up -d --build
docker compose logs -f SERVICE
docker compose exec SERVICE sh
docker compose run --rm SERVICE CMD
docker compose down -v  # removes volumes
```

GIT: common recovery

```
git status -sb
git log --oneline --graph --decorate -n 20
git add -p
git commit --amend --no-edit
git restore --staged FILE
git restore FILE
git reset --soft HEAD~1   # keep changes
git reset --hard HEAD~1   # lose changes
git revert SHA            # undo via new commit
git stash push -m "wip"
git stash pop
git reflog               # find lost commits
git rebase -i HEAD~N
```

TEXT / LOGS

```
rg "ERROR" path/
tail -f app.log | rg "WARN|ERROR"
awk '/PAT/ {print $1,$5}' file.log
awk -F',' '{print $2,$4}' file.csv
sed -n '10,20p' file
sed -E 's/foo([0-9]+)/bar\1/g' file
cut -d' ' -f1,3- file
sort | uniq -c | sort -nr
```

FIND / REPLACE

```
find . -name '*.go' -print
find . -type f -name '*.log' -mtime +7 -delete
find . -name '*.tmp' -delete
rg -n "TODO" -S .
rg -l "old" | xargs sed -i'' -E 's/old/new/g'   # mac
rg -l "old" | xargs sed -i -E  's/old/new/g'    #
  linux
```

JSON (jq)

```
jq '.' file.json
jq -r '.items[] | .id' file.json
jq '.items | map(.id) | unique | sort' file.json
curl -sS URL | jq -r '.data[0].name'
```

HTTP / NET

```
curl -sS URL | head
curl -i URL                          # status + headers
curl -L -O URL                       # follow +
  download
curl -X POST -H 'Content-Type: application/json' -d
  '{"a":1}' URL
curl -w '\n%{http_code} %{time_total}s\n' -o /dev/null
  -sS URL
dig +short example.com
ss -lntp | rg ':3000'                # listening ports
  (linux)
lsof -nP -iTCP:3000 -sTCP:LISTEN     # mac
nc -vz HOST 443                      # port test
openssl s_client -connect HOST:443 -servername HOST
  </dev/null
```

SSH / FILES

```
ssh user@HOST
ssh -L 8080:localhost:80 user@HOST  # local tunnel
ssh -R 9000:localhost:9000 user@HOST # reverse tunnel
scp -r DIR user@HOST:/tmp/
rsync -avz --progress DIR/ user@HOST:/path/
```

KUBECTL (k8s)

```
kubectl get pods -A
kubectl describe pod POD
kubectl logs -f POD -c CONTAINER
kubectl exec -it POD -c CONTAINER -- sh
kubectl port-forward svc/SVC 8080:80
kubectl apply -f file.yaml
kubectl rollout status deploy/NAME
kubectl rollout undo deploy/NAME
```

SYSTEMD / LOGS (linux)

```
systemctl status SERVICE
systemctl restart SERVICE
journalctl -u SERVICE -f --since "10 min ago"
journalctl -xe  # last errors
```

ARCHIVES / HASH

```
tar -czf out.tgz DIR/
tar -xzf out.tgz
zip -r out.zip DIR/
unzip out.zip -d out/
sha256sum file        # linux
shasum -a 256 file    # mac
```

PROCESSES / PORTS

```
ps aux | rg NAME
kill -TERM PID
pkill -f PATTERN
```

QUICK LOCAL SERVERS

```
python -m http.server 8000
ruby -run -e httpd . -p 8000
npx http-server -p 8000 .
```

TMUX (prefix: Ctrl-a)

```
tmux new -s NAME
tmux a -t NAME
tmux ls
tmux kill-session -t NAME
Ctrl-a  "  split vertical
Ctrl-a  %  split horizontal
Ctrl-a  o  next pane
Ctrl-a  d  detach
```

Tip: keep this in your dotfiles; edit + reprint as your stack evolves.